Machine learning Project
(Natural Language Processing with Disaster Tweets)

Saeed Soufeh
saeed.soufeh@studenti.unipd.it

1. Introduction

Natural language processing (NLP) is a subfield of linguistics, computer science, and artificial intelligence concerned with the interactions between computers and human language, in particular how to program computers to process and analyze large amounts of natural language data. The goal is a computer capable of "understanding" the contents of documents, including the contextual nuances of the language within them. The technology can then accurately extract information and insights contained in the documents as well as categorize and organize the documents themselves [1]. This machine learning project is about detecting disaster tweets via NLP. Sometimes people use Twitter to share live news about disasters happening in front of their eyes, and authorities can become aware of these disasters sooner by reading people's tweets.

2. Dataset

There are about 7,600 tweets in the training set labeled as 0 and 1 (non-disaster and disaster tweets, respectively) and 3,300 tweets in the test set. The training set has columns such as "id", "keyword" and "location". This information is not useful in training process, so these columns are deleted. In the text column, 3,270 tweets are labeled as 1 in the target column, and 4,350 tweets are labeled as 0. Hence, the dataset is reasonably balanced, and there is no strong need to use undersampling or oversampling. Also, in NLP words should be converted to some vectors, and the vectorized data has no specific numerical range to see whether it is needed to use feature scaling on them or not.

In this phase, it would be necessary to do some data preparation. There are many useless words, symbols, URLs, punctuation marks, etc., which make the feature space larger without adding useful information. Therefore, the data should be cleaned by replacing them with blank space.

3. Method

Different machine learning algorithms were used to compare performance:

Logistic Regression: This is one of the simplest classification algorithms to implement. I use different hyperparameters to see which regularization value works better.

SVM: The Support Vector Machine algorithm can provide strong results in NLP tasks. In this particular problem, we are facing a lot of features and it means overfitting is more likely. Using SVM helps us to avoid overfitting. It is computationally more expensive than Logistic Regression but it finally gives good accuracy.

K-Nearest Neighbors (KNN): I use this algorithm to test which value of K gives the best accuracy for this problem and to compare it with other algorithms.

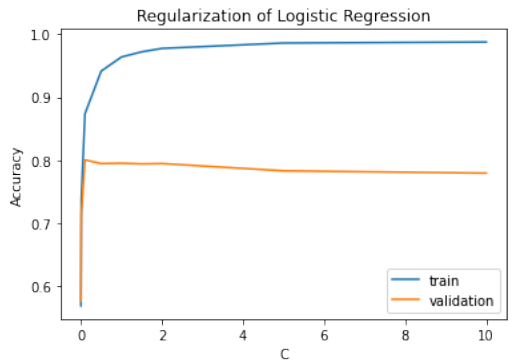
Neural Network: This algorithm is complex and needs time and computation to find the best model. But at the end it gives a good accuracy.

Decision Tree: I use this algorithm with different parameters to see its accuracy with the best parameters.

4. Experiment

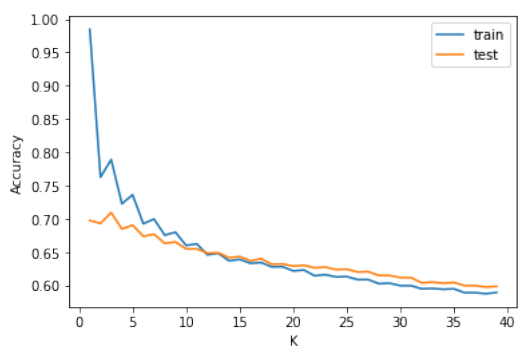
In this section, I explain what I did for each algorithm and report the best validation accuracy:

4.1. Logistic Regression: First, a simple Logistic Regression algorithm was implemented and then different regularization values were tested for the best results. C = 0.1 gave the best regularization behavior. Finally, the best accuracy was 79.19%.

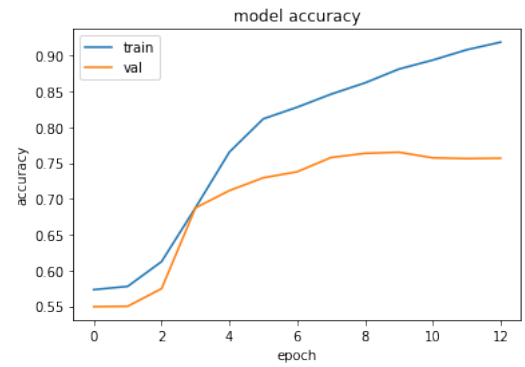


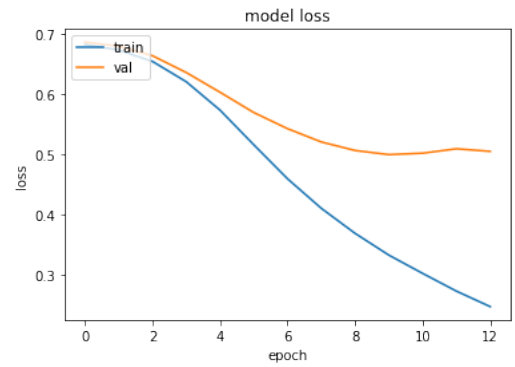
4.2. SVM: First, an SVM with a linear kernel was used, and it gave 73.58% validation accuracy. However, by using an RBF kernel and testing different values of C and gamma, the best setup was C = 1 and gamma = 0.1 and it gave 79.19% accuracy as the best one. To have this algorithm run faster, I reduced the number of features based on changing the minimum amount of repetition of every word (set min_df equal to 3).

4.3. KNN: The algorithm was tested with different values of K. However, no good results came out of this algorithm and the best accuracy is not more than about 70%.



4.4. Neural Network: I used different epochs to see the best result on Neural Network. However, I set early stopping to just run the algorithm in an optimal way and suitable epoch was 13. I trained the model and achieved 76.52% accuracy on validation set. But unfortunately, I could not predict the labels for the test set to see its accuracy (I found errors in my code which I wasn't able to debug)






```
Epoch 5/50
357/357 [==============================] - 2s 7ms/step - loss: 0.5737 - accuracy: 0.7655 - val_loss: 0.6035 - val_accuracy: 0.7117
Epoch 6/50
357/357 [==============================] - 2s 6ms/step - loss: 0.5159 - accuracy: 0.8119 - val_loss: 0.5697 - val_accuracy: 0.7295
Epoch 7/50
357/357 [==============================] - 2s 6ms/step - loss: 0.4593 - accuracy: 0.8282 - val_loss: 0.5428 - val_accuracy: 0.7379
Epoch 8/50
357/357 [==============================] - 2s 7ms/step - loss: 0.4102 - accuracy: 0.8464 - val_loss: 0.5207 - val_accuracy: 0.7579
Epoch 9/50
357/357 [==============================] - 2s 6ms/step - loss: 0.3685 - accuracy: 0.8621 - val_loss: 0.5065 - val_accuracy: 0.7637
Epoch 10/50
357/357 [==============================] - 2s 6ms/step - loss: 0.3323 - accuracy: 0.8814 - val_loss: 0.4997 - val_accuracy: 0.7652
Epoch 11/50
357/357 [==============================] - 2s 6ms/step - loss: 0.3018 - accuracy: 0.8940 - val_loss: 0.5020 - val_accuracy: 0.7574
Epoch 12/50
357/357 [==============================] - 2s 7ms/step - loss: 0.2723 - accuracy: 0.9084 - val_loss: 0.5093 - val_accuracy: 0.7563
Epoch 13/50
357/357 [==============================] - 2s 6ms/step - loss: 0.2465 - accuracy: 0.9191 - val_loss: 0.5051 - val_accuracy: 0.7568
Epoch 00013: early stopping
```

4.5. Decision Tree: There is a high difference between train and validation accuracy in this algorithm (about 25%) and which suggests overfitting and it needs more tuning to reduce overfitting. But also, I achieved 74.82% as the best score.

To summarize, Logistic Regression and SVM (with RBF kernel) gave the best accuracy (about 80%). KNN was not a good choice for this particular project. The Neural Network worked reasonably on the validation set. The Decision Tree algorithm is overfitted but still gives moderate accuracy.

References

[1] https://en.wikipedia.org/wiki/Natural_language_processing